



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Práctica 1.

Fundamentos

Silverio Martínez Andrés

Sistemas Embebidos

Grupo: 3

Semestre 2026-2

PROFESOR: Eduardo Fragoso Navarro

Fecha de entrega: 17 de febrero de 2026

Práctica 1. Fundamentos

Introducción

Los sistemas embebidos son la base de la electrónica moderna, permitiendo el control de dispositivos mediante la integración de hardware y software. Para un ingeniero en computación, es fundamental comprender no solo cómo escribir código, sino cómo este interactúa directamente con los registros físicos de un microcontrolador.

En esta práctica, nos adentraremos en el desarrollo de firmware utilizando el microcontrolador PIC16F876A. El enfoque principal consiste en establecer un flujo de trabajo eficiente, desde la escritura del código en lenguaje C utilizando el entorno MPLAB X IDE y el compilador XC8, hasta la verificación lógica mediante herramientas de simulación como SimulIDE. A través de un ejercicio clásico de contadores y visualización en un display de 7 segmentos, exploraremos conceptos esenciales como la configuración de puertos digitales (entradas y salidas), el manejo de retardos y la lógica de control mediante botones, sentando así las bases para proyectos más complejos de sistemas embebidos.

Objetivos

- Objetivo General:

- Implementar y simular un sistema básico de entrada/salida utilizando el microcontrolador PIC16F876A, para comprender el proceso de configuración de registros y la manipulación de puertos digitales mediante lenguaje C.

- Objetivos Específicos:

- Instalar y configurar correctamente el entorno de desarrollo MPLAB X IDE junto con el compilador XC8, así como el software de simulación SimulIDE, asegurando la compatibilidad entre las herramientas.
- Comprender y aplicar la configuración de los registros de control de puertos (TRIS y PORT) para definir el comportamiento de los pines del microcontrolador como entradas digitales (botón) o salidas digitales (display).
- Desarrollar un programa en lenguaje C que realice un conteo decimal (0-9) visualizado en un display de 7 segmentos, controlado por la interacción de un usuario mediante un botón pulsador.
- Analizar y corregir errores de lógica y conexión mediante la simulación del circuito, verificando la importancia de las resistencias limitadoras y la correcta asignación de pines.

Desarrollo

1. Instala el ambiente de desarrollo integrado (IDE) MPLAB® X IDE para programar el microcontrolador PIC16F876A.

El cuál fue instalado desde la misma página de MPLAB desde el siguiente enlace: <https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide> donde, después de hacer scroll a la página, se llega a la siguiente sección donde aparecen varias opciones para descargar MPLAB® X IDE, donde, en mi caso, como tengo sistema operativo Windows, descargué la opción de Windows, procediendo así con la descarga

Title	Version Number	Date	
MPLAB X IDE (Windows)	f81dbef4... eb18 6.30	23 Jan 2026	Download
MPLAB X IDE (Linux)	4caefcaa... 64a4 6.30	23 Jan 2026	Download
MPLAB X IDE (macOS)	5e7c7aec... 73a4 6.30	23 Jan 2026	Download
MPLAB X IDE Release Notes	6.30	23 Jan 2026	Download

Imagen 1. Opciones de descarga de MPLAB® X IDE

Una vez descargado, se abre el instalador, en el cual, para ahorrar espacio en disco (y como en realidad no se utilizarán todas las funciones que vienen incluidas en el software, sino que solo se utilizarán las necesarias para trabajar durante todo el curso), solo se instalará MPLAB® X IDE (No MPLAB® X IPE), y el compilador XC8 (no XC16, XC32, y otros). Quedando simplemente instalado lo siguiente:

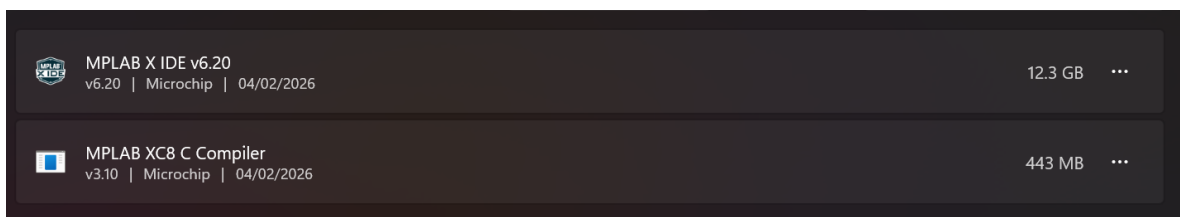


Imagen 2. MPLAB X IDE y XC8 instalado en mi dispositivo

2. Instalar la herramienta de simulación SimulIDE.

Al igual que con MPLAB® X IDE, se accedió al link de la página oficial de SimulIDE, el cual es: <https://simulide.com/p/testers/> donde aparecerán varias opciones, entre ellas la de “If you looking for table versions go here”, apretando esa opción como se muestra a continuación para así descargar la última versión estable de SimulIDE.

Tester Builds

These are builds with the last bug fixes for testing.

If you are looking for stable versions go [here](#).

Imagen 3. Opciones para descargar SimulIDE

Después de ahí, se nos conducirá a otra página en el mismo sitio, mostrando las siguientes opciones de descarga:

SimulIDE:

- **SimulIDE 1.1.0** – Last Stable version.
- **SimulIDE 1.0.0** – old Stable version.
- **SimulIDE 0.4.15** – older version.

Imagen 4. Descargas de SimulIDE

Descargando así “SimulIDE 1.1.0”, la cual es la última versión estable de SimulIDE.

3. Elabora en SimulIDE el circuito mostrado en la Figura 1.

Una vez instalado SimulIDE, se buscará replicar el siguiente circuito proporcionado por el profesor para su implementación y análisis.

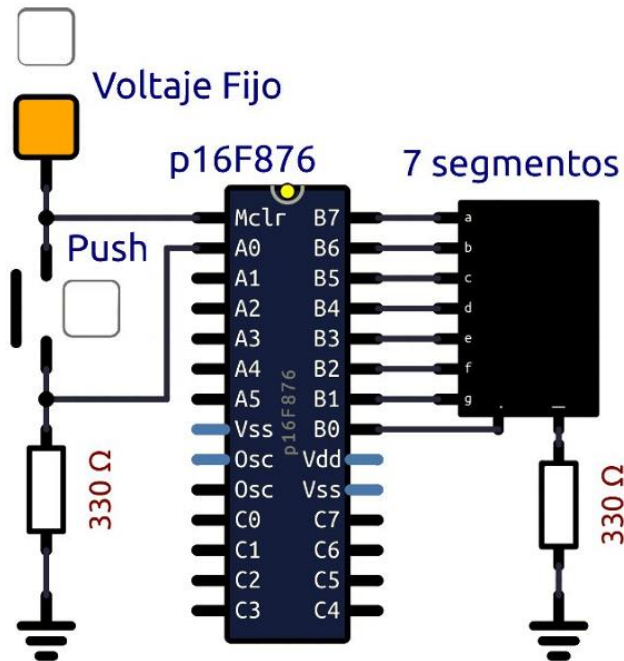


Figura 1. Circuito esquemático en SimulIDE

Para poder implementar dicho circuito, se tendrán que buscar los componentes en SimulIDE, así como realizar el cableado correspondiente al circuito que se busca replicar. En el entorno de trabajo, los componentes se encuentran en la parte izquierda, por lo que solo es cosa de seleccionar el componente deseado, y arrastrarlo donde lo queramos colocar.

Los componentes a utilizar son:

- Un pulsador
- Un voltaje fijo
- Dos tierras
- Dos resistores de 330 Ω
- Un display de 7 segmentos
- Un PIC p16F876

Quedando el circuito de la siguiente manera ya implementado en el entorno de trabajo.

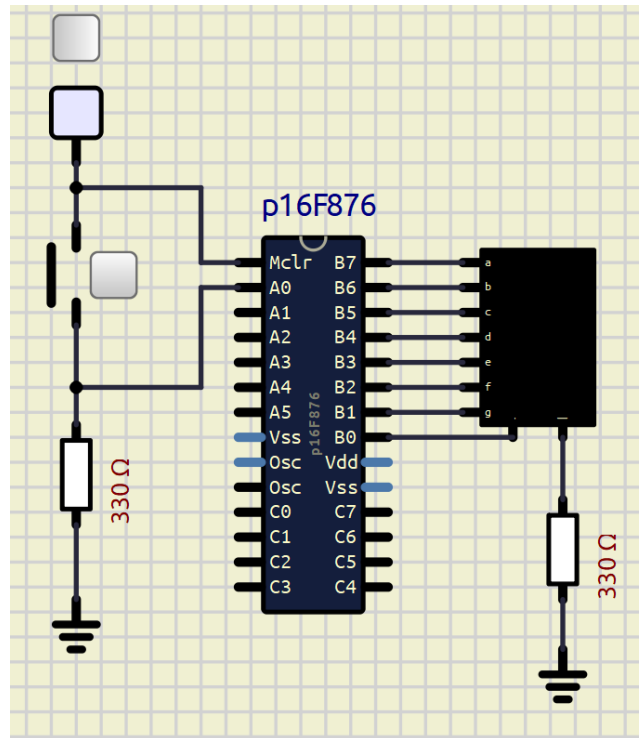


Imagen 4. Implementación del circuito de la figura 1 en SimulIDE

El circuito que armé en SimulIDE basándome en el que se busca en la práctica utiliza el microcontrolador PIC16F876A como elemento principal. Para la entrada, conecté un botón (push button) al pin RA0 configurado con una resistencia *pull-down*, de modo que al presionarlo envía un "1" lógico (5V) al microcontrolador. Para la salida visual, conecté un display de 7 segmentos al Puerto B completo (RB0 a RB7). Cada segmento del display tiene su resistencia de protección; en la simulación ajusté estos valores para que el brillo fuera el adecuado. Todo el sistema está alimentado con Vdd y Vss y utiliza su oscilador correspondiente para funcionar.

4. En MPLAB® X IDE, elabora un programa en C que cumpla con las siguientes acciones:

- Al energizar el circuito, el display debe mostrar el dígito correspondiente al valor 0 (cero)
- Mientras el botón no esté presionado, mantener el dígito del display. Por otro lado, si el botón se mantiene presionado, incrementar el valor del dígito en el display siguiendo la secuencia 0,1,2,3,4,5,6,7,8,9.
- Asignar un tiempo antes del siguiente incremento para poder observar cada uno de los dígitos durante este periodo.

Compila el programa y verifica que se haya creado el archivo .hex en la carpeta proyecto/dist/default/production

El programa en C

```

/*
 * Práctica 1 - Fundamentos de Sistemas Embebidos
 * Contador de 0 a 9 en display de 7 segmentos controlado por botó
 * Silverio Martínez Andrés
 */

#pragma config FOSC = XT
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF

#include <xc.h>

// Definir frecuencia del oscilador para la función __delay_msSi,
#define _XTAL_FREQ 4000000

// Códigos del Display de 7 Segmentos
const unsigned char display_codes[10] = {
    0xFC, // 0 (1111 1100) -> a,b,c,d,e,f ON
    0x60, // 1 (0110 0000) -> b,c ON
    0xDA, // 2 (1101 1010) -> a,b,d,e,g ON
    0xF2, // 3 (1111 0010) -> a,b,c,d,g ON
    0x66, // 4 (0110 0110) -> b,c,f,g ON
    0xB6, // 5 (1011 0110) -> a,c,d,f,g ON
    0xBE, // 6 (1011 1110) -> a,c,d,e,f,g ON
    0xE0, // 7 (1110 0000) -> a,b,c ON
    0xFE, // 8 (1111 1110) -> Todos ON
    0xF6, // 9 (1111 0110) -> a,b,c,d,f,g ON
};

```

```

void main(void) {

    // Configurar pines analógicos como digitales
    ADCON1 = 0x06;

    // Configurar direcciones
    TRISAbits.TRISA0 = 1;    // RA0 como entrada (Botón)
    TRISB = 0x00;           // Todo el Puerto B como salida (Display)

    // Inicializar variables
    unsigned char contador = 0; // Variable para guardar el número actual (0-9)
    PORTB = display_codes[contador]; // Mostrar el 0 inicial

    while(1) {

        // Mostrar el valor actual en el Puerto B
        PORTB = display_codes[contador];

        // Verificar si el botón (RA0) está presionado
        if (PORTAbits.RA0 == 1) {

            // Retardo para observar el cambio y evitar rebotes excesivos
            __delay_ms(5000); // 500ms = medio segundo

            // Incrementar contador
            contador++;

            // Si el contador pasa de 9, reiniciar a 0
            if (contador > 9) {
                contador = 0;
            }
        }
    }
}

```

El programa en C funciona mediante un ciclo infinito que está todo el tiempo verificando si el botón en el puerto A ha sido presionado. Primero, configuré los puertos y definí un arreglo con los códigos hexadecimales que hacen que se muestren los números del 0 al 9 en el display, ajustados específicamente a mis conexiones (donde RB7 controla el segmento 'a').

Dentro del ciclo while(1), si se detecta que el botón se presiona, el programa hace una pausa de 5000 ms para evitar rebotes y dar tiempo a leer el número. Inmediatamente después, se incrementa la variable contadora; si esta pasa de 9, se reinicia a 0. Finalmente, se manda el valor hexadecimal correspondiente al Puerto B para que se vea el cambio en el display.

5. En SimulIDE, carga el archivo .hex en el microcontrolador y prueba tu programa. Observa detenidamente el brillo de los leds en el display. En caso de que no tengan suficiente brillo, puedes cambiar el valor de su resistencia.

Una vez ejecutado el código y que se compilara, en donde se encuentra el archivo del proyecto, se generará un archivo con extensión .hex, el cual será cargado en la PIC p16F876, seleccionando la opción de cargar firmware al hacer clic derecho en esta misma.

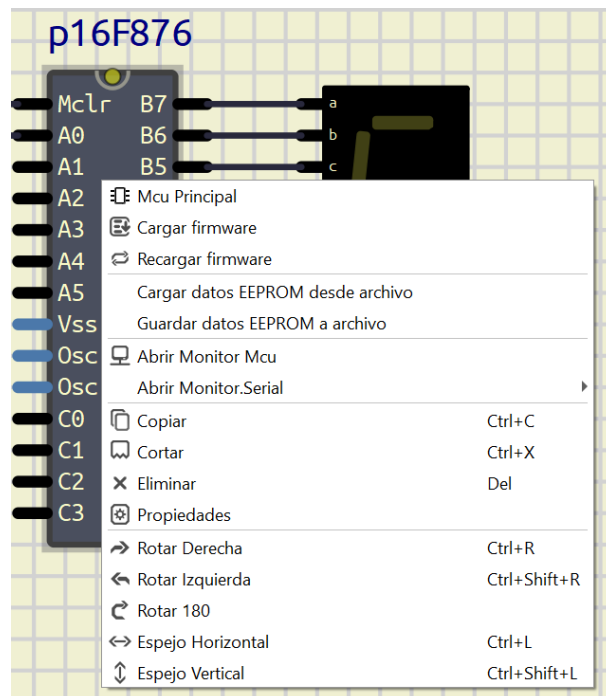


Imagen 5. Carga de firmware a la PIC p16F876

Una vez cargado el archivo .hex, solo queda correr el circuito para ver si funciona o no como se desea.

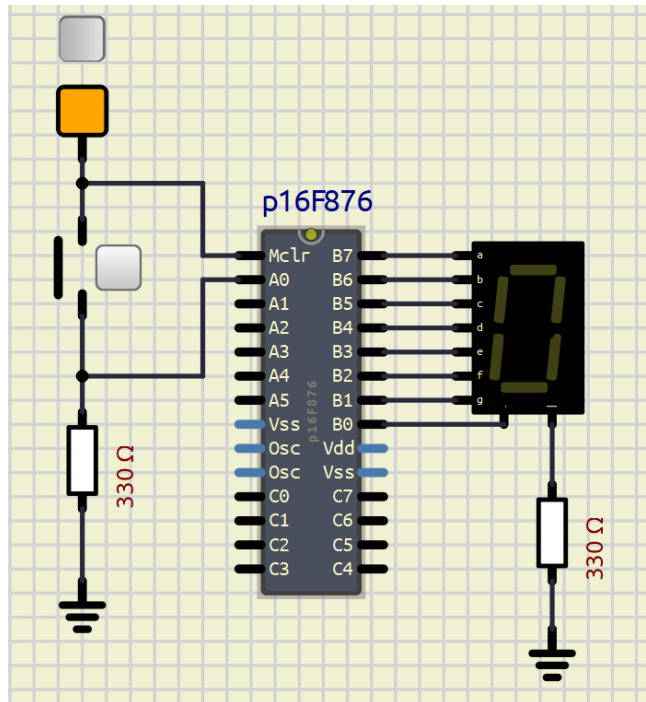


Imagen 6. Circuito implementado encendido

Al momento de correr la simulación, el display de 7 segmentos, muestra el 0 que por defecto, se buscaba implementar en el circuito.

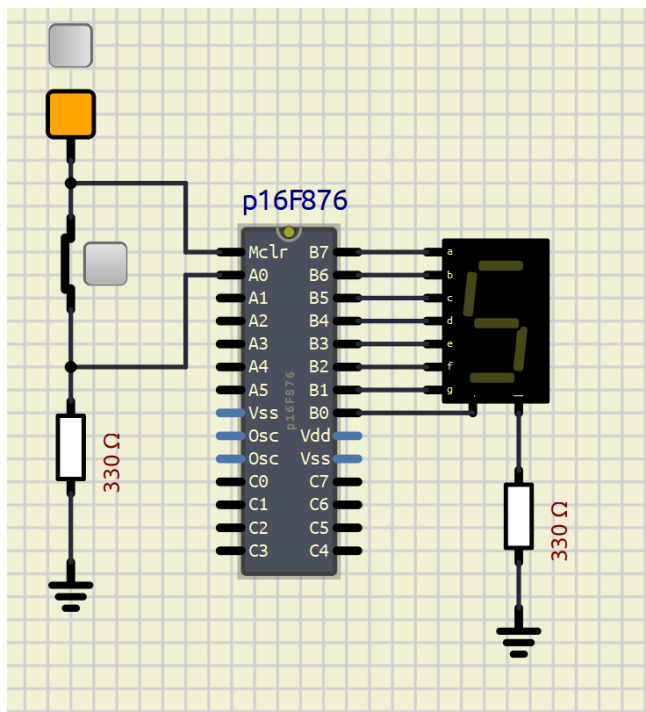


Imagen 7. Circuito implementado funcionando

Si se apreta el botón para que el pulsador haga puente y pueda pasar la corriente, el número mostrado en el display de 7 segmentos ira incrementando hasta el número 9, reiniciándose a 0 una vez pasado dicho número.

Conclusión

El objetivo principal de esta práctica fue comprender la configuración de los puertos de entrada y salida del PIC16F876A, así como el uso de registros clave como TRIS y ADCON1 para controlar el hardware. Lo que más tiempo me llevó del punto 4 fue corregir la visualización del display, ya que conecté los pines en orden inverso (RB7 al segmento 'a'), lo que provocaba que se mostraran de manera errónea; tuve que recalcular manualmente los valores hexadecimales del arreglo para adaptarlos a mi conexión física.

Para los intervalos de tiempo del punto 4.c, utilicé la función “__delay_ms(50000)” dentro de la detección del botón, logrando una pausa clara antes de cada incremento. Finalmente, observé que el brillo del display depende de la corriente que permiten pasar las resistencias; mi hipótesis es que valores altos como 330 Ω limitan demasiado la corriente en la simulación, por lo que, al reducir su valor, la intensidad de los segmentos aumenta considerablemente

Bibliografía

- [1] Microchip – MPLAB. (s.f.). <https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide>
- [2] SimulIDE – Circuit Simulator. (s. f.). <https://simulide.com/p/>